

INTRODUCTION TO HIGH PERFORMANCE COMPUTING

Matrix Multiplication using Sequential and Parallel Programs

Submitted on 17.01.2023

by group 2:

Ajay Soorya Madhav

Amritha Preetha Anil

Bryan Dunsheng See

Jiss Mariam Babu

Under the guidance of

Dr. -Ing. Habil. Peter Danielis

AIM

To carry out and compare performance and comment on the efficiency of sequential and parallel programming using matrix multiplication.

Specifically,

1. To write a sequential and a parallel program to carry out matrix multiplication of two small square matrices and print out the results
2. To develop the respective codes from task 1 to carry out matrix multiplication of two large rectangular matrices and compute the time taken for the respective programs
3. To develop a sequential and a parallel program to carry out matrix multiplication of three small matrices and print out the results
4. To develop the respective codes from task 3 to carry out matrix multiplication of three large matrices and compare the time taken and thus, performance of the two respective programs.

TASK DIVISION AND MACHINE SPECIFICATIONS

The project was divided by tasks assigned to specific group members. Task 1 was done by Jiss Mariam Babu(222100135), task 2 was done by Bryan Dunsheng See(222100181), task 3 was done by Amritha Preetha Anil(222100188), and task 4 was done by Ajay Soorya Madhav(222100147).

The tasks were formulated by the respective members. However, the observation results were compiled on a single machine of specifications:

- Architecture: x86_64
- Number of CPU cores: 16
- Model name: Intel(R) Xeon(R) Gold 6242R CPU @ 3.10GHz

THEORY

a) WHY MATRIX MULTIPLICATION:

Matrix multiplication is a very effective method for solving systems of linear equations. This branch of linear algebra is used particularly in finite difference, finite element methods, etc., which are the preliminary subjects of interest in computational engineering.

b) HOW DOES IT WORK:

Different independent, linear vectors are ordered into a list which makes up a matrix. Multiplication between these combinations of vectors is simplified with matrix multiplication. Each element of

each row of the preceding matrix is multiplied by each element of each column of succeeding matrix and are summed together to give the element of the specified row and column of the resultant matrix. This allows the subsequent matrix to contain the number of elements equal to the product of rows of the first matrix and columns of the second matrix and gives the resulting matrix an order of the rows of the first matrix and columns of the second matrix. To carry out the multiplication, however, the number of columns of the first matrix must be equal to the number of rows of the second matrix.

c) WHY PARALLEL PROGRAMMING:

As the order of the matrices increases, the computation effort and efficiency of performance also increases. As the computation of each row of the resultant matrix is independent of another, they can be computed at the same time to increase this efficiency in time.

METHODOLOGY

For the convenience of carrying out matrix multiplication with large matrices, we create a structure of a matrix with the data assigned with *malloc()*, thus, the heap of memory in lieu of stack memory. Since the heap memory is assigned hierarchically, the memory location of a particular element of the matrix needs to be specified as a combination of the indices of the row and column with respect to the number of column elements of the matrix to give the actual memory location of the element inside the heap memory. Then, inside two loops of rows and columns, values are assigned randomly.

We use *srand()* and *rand()/RAND_MAX* to assign random values between 0 and 1.

With respect to the exact memory of the elements, we carry out matrix multiplication sequentially.

With respect to the exact memory of the elements, we carry out matrix multiplication parallelly. We initialize a parallel region with *#pragma omp parallel* and create for loops with the *for collapse(number of loops to be parallelized)* clause. The threads carry out the independent multiplications and since, only the first line under for clause is parallelized, the sum of the elements to give the resultant matrix is carried out sequentially.

For setting the number of threads, we use *omp_set_num_threads()* and we get the maximum threads available with the machine with *omp_get_max_threads()*.

We find the time with the help of clock speed by setting a counter before multiplication and closing it after. In the parallel code, we make use of a defined function *omp_get_wtime()*.

For double matrix multiplication, we first create a temporary matrix to store the product of the first two matrices in the heap and multiply it with the third.

OBSERVATION

An observation of the time and thus, performance is listed and a comparison is drawn.

TASKS	SEQUENTIAL (time in s)	PARALLEL (time in s)
TASK 6_2 - a/b	0.21580	0.02213
TASK 6_4 - a/b	8.355622	0.964778

Table: Exact time measured in seconds in task 6.2 a, b and task 6.4 a, b

```
Time taken to execute in seconds: 0.21580 s
Time taken to execute in seconds: 0.02213 s
Time for computation using parallel program is lesser than sequential by 9.75157
6 times
```

Fig. Time computed from task 6.2 a, b

```
The times for sequential program are:
The time taken for multiplication A*B is =4.762057 seconds
The time taken for multiplication of (A*B)*C is =4.590347 seconds
The total time taken for multiplications is =9.352404 seconds

The times for parallel program are:
The time taken for multiplication A*B is =0.515779 seconds
The time taken for multiplication of (A*B)*C is =0.465559 seconds
The total time taken for multiplications is =0.981338 seconds

The parallel program is 9.530253 times faster than sequential code.
With matrices of higher order parallel programming is better.
```

Fig. Time computed from task 6.4 c

CONCLUSION

When the matrix order is high then parallel programming is better. It is observed from our project that parallel programming for large size matrices is approximately 9.530253 times faster than sequential programming.

REFERENCES

- Lecture notes under course “Introduction to High Performance Computing”
- <https://stackoverflow.com/search?q=openmp>
- <https://www.openmp.org/spec-html/5.0/openmp.html>
-  Introduction to Parallel Programming